

Design specification – Software Design

Document ID	SDS-SPEC-001
Revision	8569a89
Date	2026-10-03
Status	Controlled in git

Contents

1 Design inputs	3
2 Purpose	4
3 Design Outputs	5
3.1 Dynamic view	9
4 Commands and events	10
5 Dependencies	11
6 Out of scope	11

Id: SDS-SPEC-001 · Kind: design · Context: specification · Realises: DI-61, DI-63, DI-70

1 Design inputs

- **DI-1** — RDM shall read the user-need registry and the design inputs that trace to it from the frontmatter of the record, with no project-management dependency: reading the record imports no planning tool and needs no planning directory.
 - *Traces to:* UN-001, UN-004
- **DI-31** — RDM shall discover verification tags in non-Python test sources — JavaScript/TypeScript allure.story calls and Java @Story annotations across conventional test-file names — so tag-linkage warnings and verification work in polyglot repositories; only the story names a design input.
 - *Traces to:* UN-004
- **DI-40** — RDM shall read a Python test file's verification tags only from allure story decorators on its test functions and classes and from a module-level pytestmark — never from strings or comments — so a test that writes fixture files is not counted as verifying the ids those files name; a file that does not parse falls back to the decorator pattern. Only the story names a design input: feature and epic carry the bounded context and the user needs.
 - *Traces to:* UN-004
- **DI-2** — The design gate shall fail until the design documents, the documents that declare user needs, the risk documents and the design review are present, complete (no placeholder) and approved (committed) in git; a later edit re-opens it, and outside git it passes saying approval could not be checked.
 - *Traces to:* UN-002
- **DI-71** — RDM's git hooks shall run the design gate on every commit and every merge that stages implementation files, whatever their names hold, whatever the case of their extension and however they are staged (added, changed, renamed, copied or changed in type), and block it unless the gate passes or when the gate cannot run; a commit of only design documents passes.
 - *Traces to:* UN-002
- **DI-26** — rdm hooks shall install only the design-gate hooks by default — the pre-commit hook, and a pre-merge-commit hook that runs the same gate on a merge — adding the issue-reference hooks solely when requested via an explicit flag.
 - *Traces to:* UN-002
- **DI-46** — The design gate shall fail when a user-need or design-input id is declared more than once across the DHF — twice in one document or in several — naming every document that declares it; the graph shall record how many times each user need and design input is declared, and its shapes shall report a repeated declaration as a violation. The design gate shall also fail on a declaration the record reader cannot read and on a Markdown document of the DHF whose frontmatter cannot be read, naming the document, and warn on a context with more than one design document.
 - *Traces to:* UN-007
- **DI-15** — RDM shall scaffold a new documentation project from one command into a directory that does not yet exist, laying down the document templates (including the design-controls

set and a V&V plan with an empty user-need registry), the agent workflow runbook, build Makefile, container build and render config, buildable offline without an RDM checkout, and naming what it laid down and the next steps.

- *Traces to:* UN-008
- **DI-22** — RDM shall scaffold a new design input: allocate the next unused DI id, insert the entry into the chosen context's design_inputs frontmatter, keeping the document's other content, comments and line endings, emit a stub acceptance test tagged with the new id, with a component label to fill in, that fails until implemented and shadows no test module elsewhere in the suite, and print the remaining traceability checklist, rejecting, with the record left as it was, an unknown context or user need, a context with more than one design document, or a design_inputs list it cannot extend in place.
 - *Traces to:* UN-010
- **DI-24** — RDM shall bring an existing repository under design controls from one command: lay down the DHF skeleton (V&V plan, per-context design template, design review, traceability matrix, render configuration), the agent workflow runbook, the design-gate hooks, a session bootstrap, a CI gate workflow and a .gitignore of what RDM generates, skipping (never overwriting) any destination that already exists, a symbolic link included, and naming the .gitignore lines to add when it keeps an existing one.
 - *Traces to:* UN-011
- **DI-5** — RDM shall classify AI-persona simulated-use runs into a per-user-need formative status (clean / issues / failed / not_run), failed over issues over clean across a need's runs, and report, never count as clean, a run file it cannot read as a run or a run naming a user need the registry does not hold.
 - *Traces to:* UN-005
- **DI-33** — RDM shall ingest per-user-need validation records (user need, disposition, reviewer) from the DHF's validation directory and report, at the release gate, each user need lacking an approved validation record (an approved disposition with a named reviewer) as a warning that does not block release.
 - *Traces to:* UN-005

2 Purpose

The core bounded context: the user needs, the design inputs that refine them (each owned by one context and naming the needs it traces to), the tagged tests that claim to verify them, the design review, and the design gate that blocks implementation until the record is complete, approved and declares every id once. It speaks the glossary's user need, design input, design document, tagged test, design review, approved and design gate, and, for validation, validated (validation records, formative persona evidence). Every other context conforms to its ids. Onboarding (`rdm init`, `rdm adopt`), the commands that create the record, is not a context: it is this context's application layer.

3 Design Outputs

The design outputs are this context's architecture, in the C4 model of the architecture workspace: its components, what each is responsible for, and how they relate. They name components, never code: the workspace maps each component to its code.

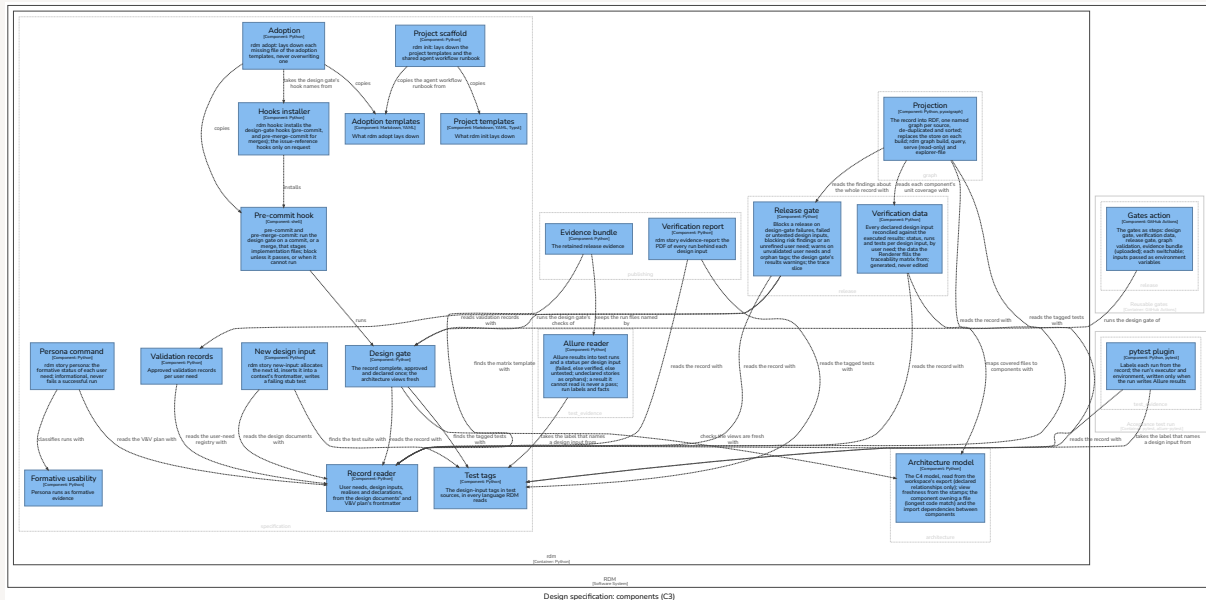


Figure 1: Components: specification

Component	Responsibility	Meets
Record reader	Reads the user needs, the design inputs (a user need traced as one value, not a list, is that one need), what each context realises, and every declaration of an id, from the frontmatter of the design documents and the V&V plan. Finds a design document by its <code>kind: design</code> marker, never by its name; an id declared more than once keeps its first declaration, and every declaration is listed. Lists every declaration it cannot read (design inputs that are not a list of entries each with one id, a user need with no id, design inputs outside a design document), so none is dropped unseen.	DI-1, DI-46
Test tags	Finds the test suite (no higher than the DHF's repository root) and the design input each test claims to verify: in Python from the syntax tree, on tests only (never a helper) and under any name the file imports the story decorator as, never from	DI-1, DI-31, DI-40

Component	Responsibility	Meets
	strings or comments, falling back to the decorator pattern when a file does not parse; in JavaScript, TypeScript and Java by pattern, across the conventional test-file names; and in a YAML task's own tags. Only the story names a design input. Lists each tagged test by name for the knowledge graph.	
Design gate	Fails unless at least one design document and the design review exist, hold no placeholder markers (each a whole word), and are committed clean, as must every document that declares user needs and every risk document (the register and the policy); while a merge is being made, a document staged exactly as a commit holds it at its path is committed: tracked, with no uncommitted edit and none hidden from git, and, for a link, its target too; an edit re-opens it, and outside git approval is reported as unverifiable and the verdict says so. Fails on an id declared more than once, naming every document that declares it, on a declaration the record reader cannot read, on a Markdown document of the DHF whose frontmatter cannot be read (not UTF-8, not YAML, a repeated key, not a mapping, or no closing fence), and on stale or uncommitted architecture views. Warns on a user need nothing traces to, an unknown traced or realised id, a design input no test is tagged with, and a context with more than one design document.	DI-2, DI-46
Pre-commit hook	Blocks a commit that stages implementation files (source in any common language, template or configuration outside the DHF and the planning directory, whatever characters their names hold and whatever the case of their extension, added, changed, renamed, copied or changed in type) unless the design gate passes; its pre-	DI-71

Component	Responsibility	Meets
	merge-commit twin blocks a merge the same way. A commit of only the design documents passes: that commit is the approval. When the gate cannot be run, the commit is blocked; an explicit, discouraged override bypasses it.	
Hooks installer	<code>rdm hooks</code> : installs the pre-commit hook into the repository's git hooks (or a given directory), and the issue-reference hooks only when explicitly asked.	DI-26
New design input	<code>rdm story new-input</code> : allocates the next unused id (one no design input declares and no test is tagged with), inserts the entry into the chosen context's frontmatter by a targeted edit that keeps its comments and line endings (and refuses, leaving the document as it was, when the edit would not read back as the same inputs plus the new one), writes a stub acceptance test tagged with the id, with a component label to fill in, that fails until implemented (in <code>test_<context>.py</code> , or <code>test_<context>_acceptance.py</code> when a test module of that name exists elsewhere in the suite), and prints the remaining traceability checklist. Refuses an unknown context or user need, and a context with more than one design document. It can also list the contexts, the taken ids, the next id and the user needs.	DI-22
Project scaffold	<code>rdm init</code> : copies the project templates into a new directory, and the agent workflow runbook from the adoption templates, so both scaffolds share one runbook; refuses a directory that exists (exit 2), and on success names what it laid down and the next steps.	DI-15
Project templates	What <code>rdm init</code> lays down: the document templates (including the design-controls set), the build Makefile (a Word document is rebuilt when its reference document changes), the render configuration, the	DI-15

Component	Responsibility	Meets
	container build (installing the RDM release from git when no wheel is given), and the Pandoc and Typst configuration. Templates use only local images.	
Adoption	<code>rdm adopt</code> : creates each missing file of the adoption templates and skips, never overwrites, one that exists (a symbolic link exists, dangling or not); when a <code>.gitignore</code> exists, names the lines of RDM's generated files to add to it; keeps the hook and the bootstrap executable; stamps the installed version as its release tag (a pre-release such as 2.0.0a0 as 2.0.0-alpha, the tag the release and its image carry); reports what it skipped and prints the next steps. It copies the pre-commit hook rather than holding its own, so the gate has one source.	DI-24
Adoption templates	What <code>rdm adopt</code> lays down: the DHF skeleton (V&V plan, the per-context design template, design review, traceability matrix, render configuration), the agent workflow runbook, a session bootstrap, a CI gate workflow and a <code>.gitignore</code> of RDM's generated files (Allure results, verification data, the graph store, bytecode).	DI-24
Validation records	Reads the per-user-need validation records from the DHF's validation directory; only an approved disposition with a named reviewer counts. Gives the user needs without one.	DI-33
Formative usability	Reconciles persona runs against the user-need registry into a formative status per need: failed if any run could not finish (a run that does not say it completed did not), else issues if problems were seen, else clean; not run when none tried. A run file it cannot read as a run is listed as unreadable, never counted clean; a run naming an unknown need is listed as an orphan. Clean is not validated.	DI-5

Component	Responsibility	Meets
Persona command	<code>rdm story persona</code> : prints the formative status of each user need in the V&V plan's registry; informational, so a successful run always passes.	DI-5

The shared kernel is drawn in this view but is not this context's: every context may use it, and the arrows into it are left out of the other views. The view also draws the architecture model (`architecture`), which the design gate uses, and the components of other contexts that use this one's.

- The pre-commit hook runs the design gate; the hooks installer installs the hook.
- The design gate reads the record with the record reader, finds the tagged tests with the test tags, and checks the views are fresh with the architecture model.
- New design input reads the design documents with the record reader, so it and the gates share one view of the record, and finds the test suite with the test tags.
- Project scaffold and adoption copy their templates; the scaffold also copies the runbook from the adoption templates, and adoption the pre-commit hook.
- The persona command classifies runs with formative usability and reads the V&V plan with the record reader; validation records read the registry with it.

Realised here for other contexts: the test tags' list of tagged tests by name is this context's part of `graph`'s DI-61, and the design gate's view check is its part of `architecture`'s DI-70. The CI workflow that adoption lays down calls `release`'s reusable workflow (DI-63).

Realised elsewhere: `release` reports, at the release gate, the user needs the validation records give (DI-33); `publishing` renders the needs and design inputs the record reader reads (DI-1); and the knowledge graph records each id's declaration count and reports a repeated declaration with its shapes (DI-46). With results, the design gate's warnings about executed results are `release`'s, handed to the gate's output by the composition root: this context never reads results.

Assumption: persona runs are produced by the usability-persona agent skill, outside RDM's package; this context only reads them.

3.1 Dynamic view

The order matters for a contributor: git runs the pre-commit hook, which runs the design gate; the gate reads the design documents and the review, checks the architecture views are fresh, and finds the tagged tests. Any one failing stops the commit.

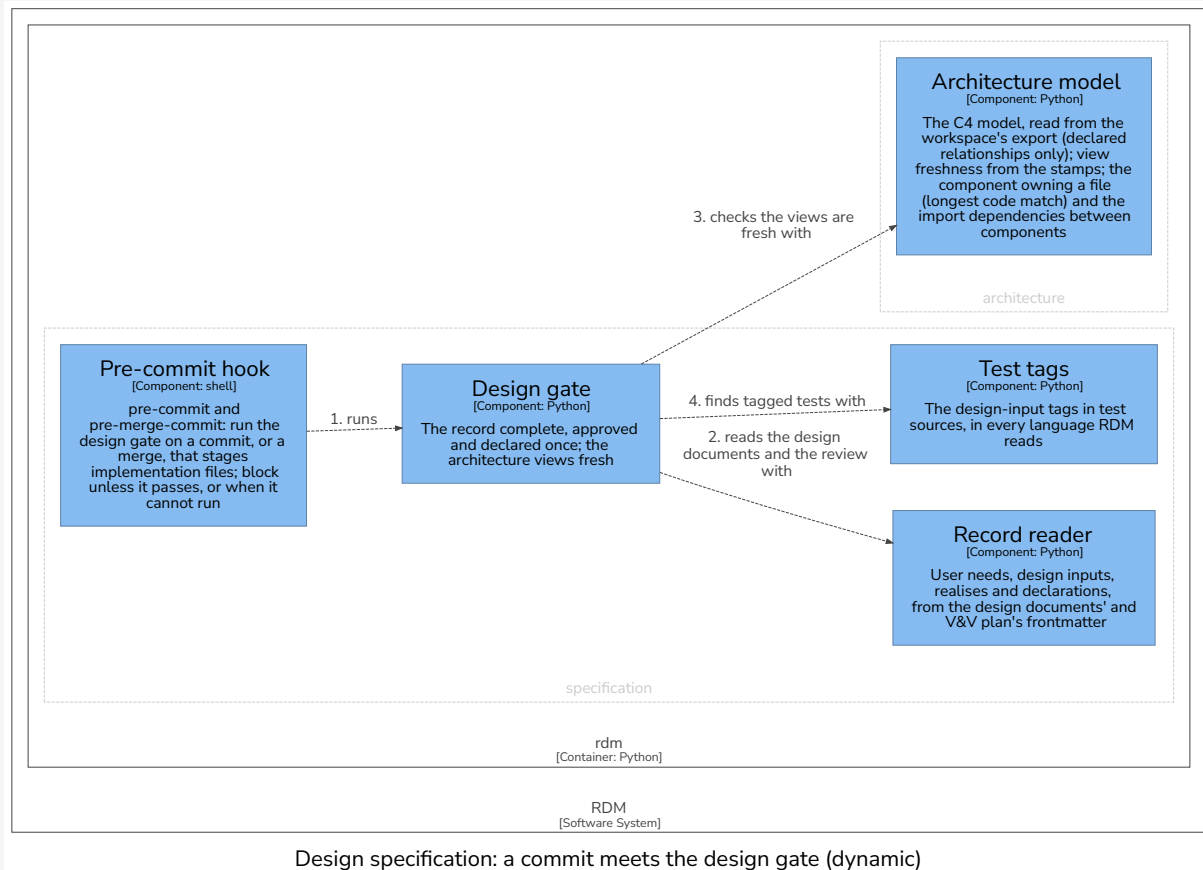


Figure 2: Scenario: a commit meets the design gate

4 Commands and events

Each row: the actor issues the command, resulting in its success event or a fail event (the rule broken, after the slash), which affects the entity.

Actor	Command	Success event	Fail events	Entity
Contributor	Declare design input	Design Input Declared	Not Declared / Unknown Context · Unknown User Need	Design document
Contributor, or the commit hook	Check design controls	Design Controls Approved	Not Approved / Uncommitted · Placeholders · Duplicate Id · Malformed Declaration · Unreadable Frontmatter · Views Stale	The record

Actor	Command	Success event	Fail events	Entity
Persona (an agent)	Reconcile persona runs	Formative Runs Reconciled	—	User need

Reactions: whenever a design input is declared, a failing stub test is written; whenever a commit is made, the design controls are checked.

5 Dependencies

Layer 2 of the dependency rule. Depends on the shared kernel and on one leaf below it: `architecture` (the design gate's view check). Depended on by `test_evidence` (the pytest plugin reads the design inputs), `release` (the release gate, the verification data), `publishing` (the DMR index, the verification report, the evidence bundle) and `graph` (the projection). The composition root wires its commands and hands the design gate `release`'s results warnings. It reads no results and no risk register.

6 Out of scope

- Summative validation. Persona evidence is formative only, not summative IEC 62366 validation, and never gates release: the release gate does not read persona runs (a structural property, not mutation-testable). The human summative study is the validation record.
- Planning. RDM ships no planning tooling; tasks live outside the record.